Lab 4 Report

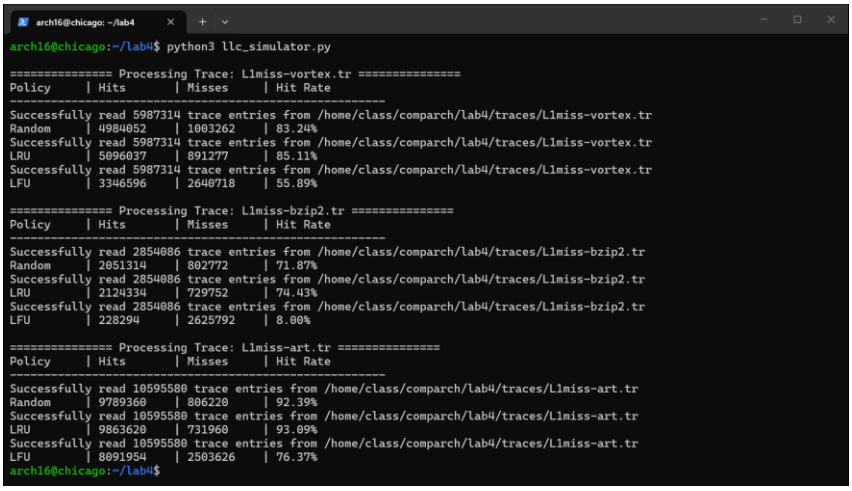
Compile and Run Commands

Run from command line using the python3 command to explicitly specify that the script should be executed using the Python 3.x interpreter.

Command line: python3 llc_simulator.py

Comparison of LLC Accuracies

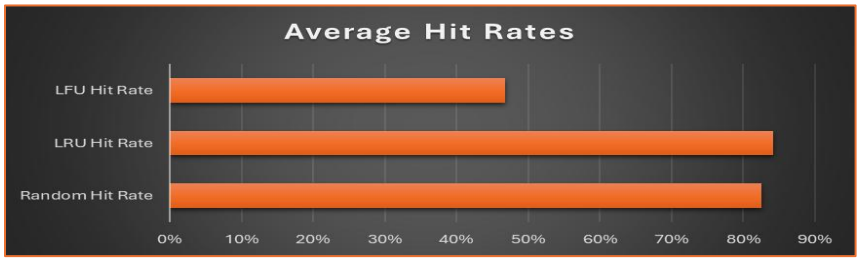
Command Line Results:



Results Comparison Table

Simulation Results Summary				
Trace File	**Total Entries**	**Random Hit Rate**	**LRU Hit Rate**	**LFU Hit Rate**
L1miss-vortex.tr	5,987,314	83.24%	**85.11%**	55.89%
L1miss-bzip2.tr	2,854,086	71.87%	**74.43%**	8.00%
L1miss-art.tr	10,595,580	92.39%	**93.09%**	76.37%

Graph of Average Success Rate by Type



Analysis of the Results

When analyzing the performance of the Level-2 Cache (LLC) simulator based on the data, several architectural observations can be made:

Least Recently Used (LRU): Best Results

Across all three trace files, LRU consistently yielded the highest hit rate (peaking at 93.09% on the art trace). This confirms that the simulated workloads exhibit temporal locality, data that was accessed recently has a high probability of being accessed again in the near future. By keeping the most recently used blocks and evicting the oldest ones, LRU perfectly caters to this programming behavior.

Random: Good Results

The Random policy performed surprisingly well, trailing LRU by only a small margin (1% to 3%). A Random policy is highly desirable because it requires no overhead counters or timestamp comparison logic. This data proves that for these specific traces, a massive amount of physical hardware complexity could be saved by utilizing a pseudo-random replacement policy with only a minor penalty to the overall hit rate.

Least Frequently Used (LFU): Lagging Results

LFU struggled in these simulations, most notably dropping to an 8.00% hit rate on the bzip2 trace. Likely because with LFU, if the application accesses a specific set of addresses thousands of times in a loop early in the trace and then moves on to a completely different part of memory, the old addresses can permanently pollute the cache. Additionally, they have high frequency counts, so the cache refuses to evict them, forcing a continuous stream of misses for the new data.

Final Assessment:

While these results were good, we could see a significant improvement by shifting to 512K Cache Size, which would address capacity misses experienced by all 3 replacement policies but, most notably for LFU where we could at least partially mitigate the effects of cache pollution.

Conversely, increasing associativity for the given traces would likely not show a significant improvement in our results (1%-2%) as our 4-way configuration seemed sufficient to prevent the majority of conflict misses based on our good results.

All things considered, utilizing a random replacement policy may be the way to go, at least with this data, given the low overhead required to achieve strong results.